



ENGINEERING CASE STUDY · COMPANY ARCHIVE

Advance Deposit Tracker

From a Shared Excel Sheet to a Secure, Role-Based
Enterprise Web Platform

Supplier advance-deposit management for Sunshine —
requests, approvals, payments, shipment risk & analytics

Document type: Technical Case Study & System Reference

Version 1.2 · July 2026

Status: Production — live on Railway (Singapore) + Supabase (Mumbai)

Table of Contents

1	Executive Summary
★	System Highlights — At a Glance
2	Background — The Excel Problem
3	Solution Overview
4	System Architecture
5	Technology Stack
6	Core Features by Role
7	Request Lifecycle & Workflow Engine
8	Data Migration from Excel
9	Security & Data Integrity
10	Performance Engineering
11	Reliability & Error Handling
12	Analytics & Business Intelligence
13	Excel vs. Platform — Outcome Comparison
14	Appendix A — Delivery Timeline
15	Appendix B — Glossary

1 Executive Summary

Sunshine's merchandising and accounts teams managed supplier **advance deposits** — partial payments made to suppliers before goods are shipped — through a single shared Excel workbook. As transaction volume grew past several hundred live records, the spreadsheet became the bottleneck for the entire payment operation: no access control, no approval workflow, no audit trail, manual analytics, and no protection against overwriting a colleague's edits.

The **Advance Deposit Tracker** replaces that workbook with a production web platform: a role-based dashboard system for five distinct user roles, a public submission form for merchandisers in the field, an enforced status workflow with a Head-of-Merchandiser approval gate for risky suppliers, automated shipment-risk analytics, one-click report exports, a complete audit trail, and an AI-powered business-intelligence assistant.



Headline result: the public form's heaviest API call dropped from **6.5 seconds to 1.8 seconds**, and single-query endpoints from **2.9 seconds to under 0.8 seconds**, after a systematic performance program: relocating compute to the database's region, cutting database round trips per request by more than half, response compression, HTTP caching, and instant-feedback (optimistic) UI updates.

★ System Highlights — At a Glance

What was delivered, by the numbers:

25 Production releases shipped	13 Client change requests delivered	601 Excel records migrated, zero loss	5 Role-based dashboards
40+ Built-in guard rails the system enforces	3 Independent enforcement layers (UI · API · database)	~72% API latency cut in the performance programme	30 min Max age of any analytics figure — recomputed automatically
128 At-risk suppliers surfaced automatically on day one	533→33 Payment-terms chaos consolidated to a curated catalogue	100% Of changes captured in the field-level audit log	18 Columns in one-click Excel / CSV / PDF exports

Capability highlights

- **A financial control tower, not a tracker.** Every advance is followed from intake to shipment with live risk classification, cost-of-fund exposure computed to the day, and a Non-Performing-Assets panel for leadership.
- **Risk that polices itself.** Suppliers who miss their grace window are flagged automatically; every future advance to them is intercepted and routed through Head-of-Merchandiser sign-off — no human has to remember who defaulted.
- **Money movements are irreversible on purpose.** Processing a payment locks the record at the database level itself — the platform refuses post-payment tampering, even from the API.
- **Client-defined mathematics, guaranteed by tests.** The Cost-of-Fund formula was specified by the business, implemented to the letter, and its worked example is encoded as an automated test — a future code change that alters the number fails the build.
- **Ask questions in plain English.** An AI business-intelligence assistant converts natural-language questions into safe, read-only queries and explains the answer.
- **Field intake without friction.** A public form lets merchandisers submit from anywhere with no login; submissions bind to their registered account automatically and obey every risk gate.
- **Governance as configuration.** Which role sees which field, which role opens which analytics section, grace days, interest rates — all admin-editable settings, not code changes.
- **Engineered for distance.** Compute co-located with the database region, round trips halved, responses compressed, actions acknowledged instantly — sub-second interactions measured from India.
- **Fully documented for the archive.** This case study plus a companion Formula & Calculation Reference put every rule and every formula on paper for client sign-off.

2 Background — The Excel Problem

The original process ran on one workbook (the "Deposit Sunshine Tracker"), passed between merchandisers, the accounts team, and finance. Every weakness of spreadsheet-as-database applied:

Pain point	Consequence in daily operations
No access control	Anyone with the file could see or edit every column — deposit amounts, bank references, supplier risk notes — regardless of their role.
No workflow	"Approved", "on hold", "cancelled" were free-text cell values. Nothing prevented a payment being processed for a request that was never approved.
No audit trail	When a number changed, nobody could say who changed it, when, or what the previous value was.
Concurrent-edit conflicts	Two people editing at once meant lost updates and emailed copies named <code>tracker_FINAL_v3 (2).xlsx</code> .
Manual analytics	Overdue-shipment tracking, cost-of-fund exposure, and supplier default risk were recomputed by hand with formulas that silently broke as rows were added.
No supplier risk gate	Nothing stopped a new advance being paid to a supplier who had already defaulted on a previous shipment.
Free-format data	Dates, currencies, and percentages were typed inconsistently — 530+ variations of payment terms existed for what were really ~33 distinct terms.

The core requirement distilled from stakeholder sessions: *"the right people should be able to do only the right things, every change must be traceable, risky suppliers must require sign-off, and the numbers we look at must compute themselves."*

3 Solution Overview

The platform delivers four connected capabilities:

3.1 Controlled intake

- **In-app request form** for merchandisers with live validation, supplier auto-fill (fixed deposit amounts), and a curated 33-term payment-terms catalogue.
- **Public submission form** (no login required) so field merchandisers can submit against their registered e-mail; submissions bind automatically to their account.
- **Flagged-supplier interception:** selecting a defaulted supplier triggers an inline warning; the submitter may withdraw or proceed — proceeding routes the request to the Head of Merchandiser for explicit approval before it can ever reach the payment queue.

3.2 Enforced workflow

- Nine request statuses and thirteen legal transitions (hold, resume, cancel, reopen, approve, reject, process) — each restricted to specific roles and enforced in the API, in the database itself, and in the UI.

- Payment processing **locks** the record; locked records are read-only for everyone below Super Admin.

3.3 Automated analytics

- Per-request analytics snapshots: grace ETD, overdue days, payment-to-ship days, cost-of-fund exposure, and an on-time / delayed / defaulted risk classification — recomputed automatically on every relevant change and on a 30-minute scheduler.
- Suppliers are **auto-flagged** when any of their shipments exceeds the ETD grace period.
- Dashboards: KPI tiles, delay buckets, month-on-month / year-to-date trend charts, drill-downs by merchandiser / vertical / customer, and a Non-Performing-Assets panel.

3.4 Governance

- Field-level audit log (who, what, old value, new value, when) for every mutation.
- Role-scoped report exports (Excel / CSV / PDF, 18 columns).
- Admin console: user management, payment-terms catalogue, per-role field visibility, per-role analytics section permissions, system configuration.
- An AI BI assistant that translates plain-English questions into read-only SQL and summarises the result.

4 System Architecture

The system is a three-tier architecture deployed on Railway with Supabase providing PostgreSQL, authentication, and row-level security. Both application services run in Railway's **Singapore region**, co-located as closely as possible with the database in AWS Mumbai and with the India-based user base.

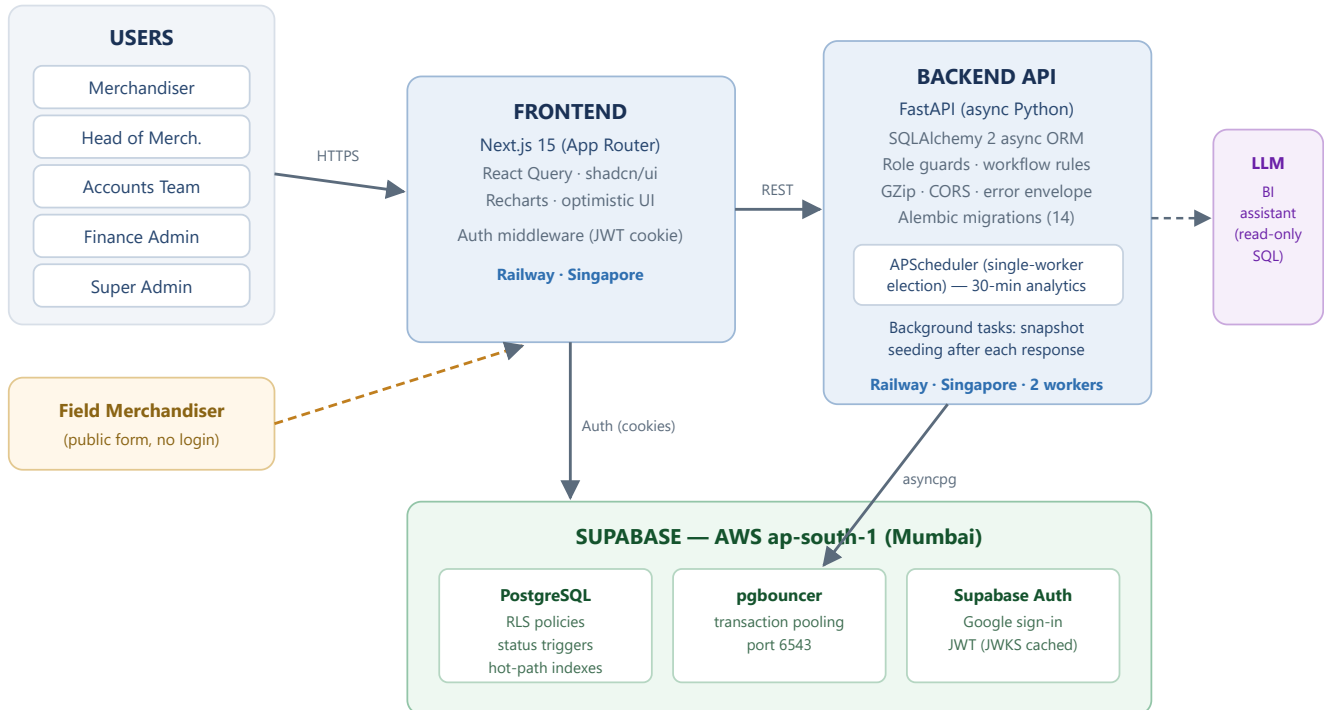


Figure 1 — High-level system architecture. All application compute is co-located in Southeast Asia; the database layer (PostgreSQL + pgbouncer + Auth) is managed by Supabase in Mumbai.

4.1 Design decisions worth recording

- **Transaction-pooled Postgres:** the API connects through pgbouncer transaction pooling; prepared-statement caching is disabled and statement names randomised — a hard requirement discovered in production (SAVEPOINTS and session-level advisory locks are also unusable through this pooler).
- **Defence in depth for workflow:** the status machine is enforced three times — Python rules (role-aware), a PostgreSQL trigger (last line of defence, role-blind), and UI gating. A write that bypasses the API still cannot make an illegal transition.
- **Single-owner scheduler:** with two uvicorn workers, a file-lock election ensures exactly one APScheduler instance runs — previously both workers ran the 30-minute analytics refresh, doubling database load.
- **Post-response computation:** analytics snapshot seeding runs as a background task after the HTTP response is sent, keeping user-facing writes fast while analytics stay near-real-time.

5 Technology Stack

Layer	Technology	Why it was chosen
Frontend framework	Next.js 15 (App Router), TypeScript	File-based routing per dashboard, server middleware for auth gating, production-grade build pipeline.
Data fetching	TanStack React Query	Cache-first navigation, polling for live queues, optimistic updates with rollback, scoped invalidation.
UI	shadcn/ui + Tailwind CSS, Recharts, Sonner	Consistent design system; composable chart primitives for the analytics suite.
Forms	React Hook Form + Zod	Declarative validation shared across the in-app and public forms.
API framework	FastAPI (Python 3.12, fully async)	Pydantic request/response validation, dependency-injected auth, OpenAPI docs for free.
ORM / migrations	SQLAlchemy 2 async + Alembic (14 migrations)	Typed models, eager-default RETURNING, versioned schema evolution run automatically on deploy.
Database	Supabase PostgreSQL (Mumbai) + pgbouncer	Managed Postgres with Row-Level Security, triggers, and a built-in auth service.
Authentication	Supabase Auth (Google SSO) + JWT verification	No password handling in-house; backend verifies tokens against cached JWKS.
Scheduling	APScheduler (single-worker election)	30-minute analytics refresh without a separate worker service.
AI assistant	LLM API (Grok) — SQL generation + summarisation	Natural-language BI over the live schema, restricted to read-only queries.
Hosting	Railway — two services, Southeast Asia region	Config-as-code (railway.toml), per-service scaling, CLI-driven region placement.
Reports	Server-generated Excel / CSV / PDF builders	Role-scoped, 18-column exports; heavy lifting stays off the client bundle.

5.1 Codebase shape

Backend: layered `api → services → repositories → models` with domain rules (`status_transitions`, `lock_rules`, `supplier_validation`) isolated from transport. Frontend: route-per-dashboard app directory, shared hooks (`useRequests`, `useAnalytics`, `useMasters`), a single axios client with a token-attaching interceptor, and role guards wrapping every dashboard page.

6 Core Features by Role

Role	What they see and do
Merchandiser	Create deposit requests (in-app or public form); auto-filled fixed deposits; flagged-supplier warning with continue/withdraw choice; own-request history with live status; hold / resume / cancel / withdraw own requests; remarks; activity feed; shipment-KPI analytics.
Head of Merchandiser	Approval queue of requests on flagged suppliers; request detail with full status history and analytics snapshot; one-click approve (→ payment queue) or reject with remarks; Non-Performing-Assets panel (overdue suppliers + merchandiser performance) with drill-downs; analytics & reports.
Accounts Team	Oldest-first pending-payment queue; payment details entry (date, bank ref, status dropdown); estimated shipment date; process payment (locks the record); hold / resume / cancel / reopen; payment analytics.
Finance Admin	Supplier risk management — flag / resolve defaulted suppliers by name with reasons; every row in the risk overview clicks through to a per-supplier drill-down (flag details, overdue stats, full request history); audit log access; full analytics including delay buckets and cost-of-fund; reports.
Super Admin	Everything above, plus: user management (roles, activation), payment-terms catalogue, per-role field-visibility matrix, per-role analytics permissions, system configuration (grace days, cost-of-fund rate), audit logs, AI assistant administration, soft-delete.

Field visibility as configuration: which columns each role can see (deposit %, invoice numbers, creator info, cost-of-fund...) is a Super-Admin-editable matrix, not hard-coded — the same request detail page renders differently per role.

7 Request Lifecycle & Workflow Engine

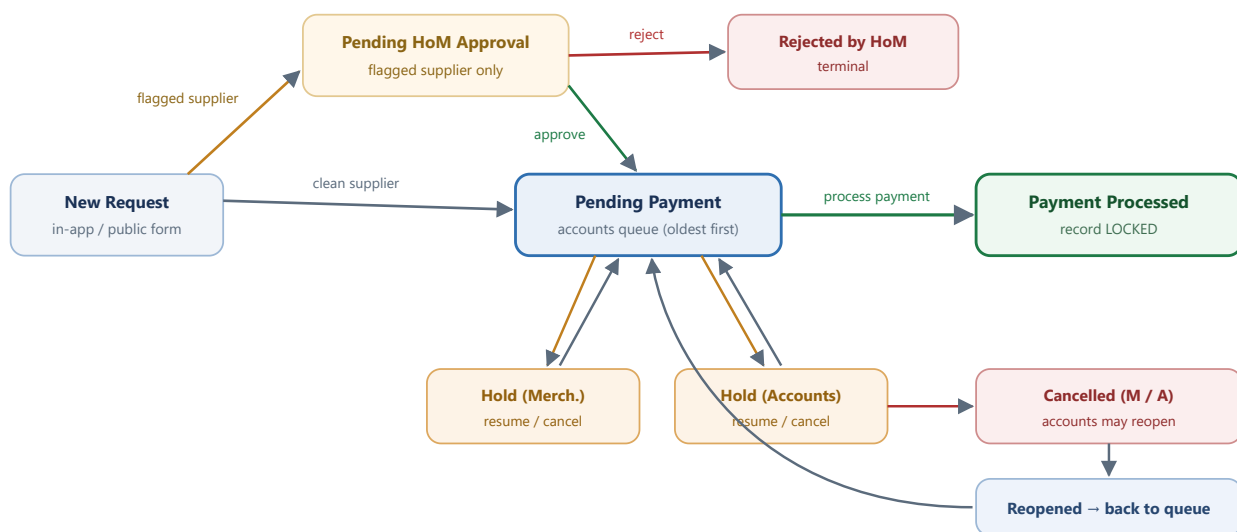


Figure 2 — The request state machine (simplified). Thirteen legal transitions; every edge is role-restricted and enforced in the API, the database trigger, and the UI.

7.1 Triple enforcement

Layer	Mechanism
Application (Python)	<code>status_transitions.py</code> — a map of <i>(from, to)</i> → <i>allowed roles</i> ; raises a domain error with a human-readable message before any write.
Database (PostgreSQL)	<code>enforce_status_transition()</code> trigger — rejects any UPDATE making an illegal transition, even from a rogue client with direct DB access.
UI (Next.js)	Buttons render only for legal actions in the current status for the current role; in-flight actions disable all controls to prevent double-fires.

Lesson learned: when the HoM statuses were added, the Python rules were updated but the database trigger initially was not — approvals failed at the last line of defence. The fix (and the convention now recorded in the runbook) is that the three layers ship together in one migration.

8 Data Migration from Excel

The legacy workbook was migrated in a single scripted pass (`backend/scripts/migrate_csv.py`) so the team started with their full history, not an empty system:

Entity	Rows	Notes
Deposit requests	601	Full history with original request dates preserved
Payment details	601	Payment dates, ship dates, references
Status history entries	601	Initial state per request
Suppliers	151	Deduplicated case-insensitively; unique index added
Customers / Verticals	25 / 18	Normalised from free-text columns
Merchandiser accounts	11	Placeholder users created from the "submitted by" column
Payment terms	533 → 33	Free-text variants consolidated into a canonical curated catalogue

Immediately after import, a full analytics recalculation built 601 snapshots and auto-flagged **128 suppliers** whose shipments had already exceeded the ETD grace period — risk exposure that had been invisible in the spreadsheet.

9 Security & Data Integrity

Control	Implementation
Authentication	Google SSO via Supabase Auth; backend verifies JWTs against cached signing keys; unregistered / deactivated accounts get explicit, friendly lockout screens.
Authorisation	Role checks at every endpoint <i>and</i> inside services; merchandisers can only act on their own records; UI role guards mirror the API.
Row-Level Security	Supabase RLS policies on all core tables as a second enforcement layer beneath the API.
Workflow integrity	Database trigger blocks illegal status transitions regardless of the caller (Section 7).
Record locking	Processing a payment sets <code>is_locked</code> ; locked records are immutable below Super Admin.
Audit trail	Every create / update / status change / delete writes an audit row: actor, entity, field, old value, new value, timestamp, user agent.
Concurrency safety	Request numbers issued under a transaction-scoped advisory lock — concurrent submissions can no longer collide; payment rows fetched with <code>SELECT ... FOR UPDATE</code> .
Privacy by design	No client IP addresses are exposed to application users or the AI assistant; the AI runs read-only SQL with an enforced row limit.
Soft deletes	Nothing is physically destroyed; deletions are flagged with actor + timestamp and excluded from queries.

10 Performance Engineering

Mid-programme, users reported multi-second waits on submits and page loads. A three-track audit (backend hot paths, frontend data-fetching, infrastructure) traced the slowness to four compounding causes — and fixed all of them in one coordinated release.

10.1 Root cause #1 — geography

The database lives in **AWS Mumbai**; the application originally ran in Railway's **US-East (Virginia)** region, with users in India. Every database query paid a ~230 ms planet-crossing round trip — and the submit flow made ~20 of them. The application was relocated to Railway's **Singapore** region (~60 ms to Mumbai), collapsing the dominant cost of every operation.

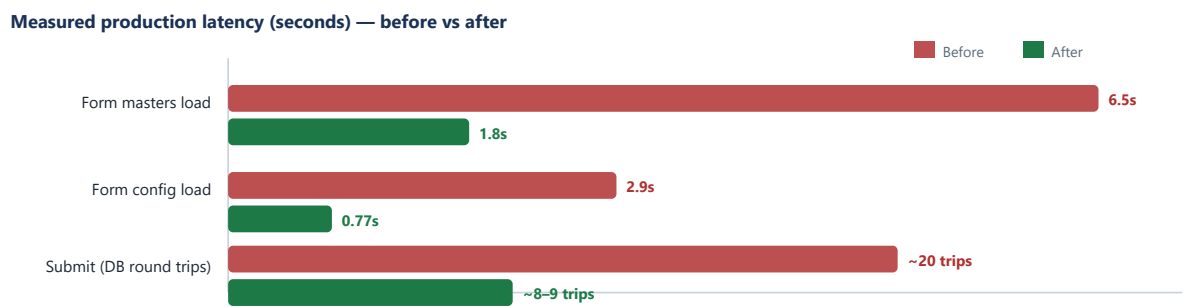


Figure 3 — Production measurements before and after the performance release (curl, three-run medians, measured from India).

10.2 Root cause #2 — database round-trip bloat

Fix	Effect
Three-tier record loading: scalar-only fetch for validation, 4-relation fetch for mutation responses, full 9-relation fetch only for detail views	Submit reload: 10 queries → 5; payment paths: –9 queries each
Server-generated defaults returned via <code>RETURNING</code> (eager defaults) instead of a follow-up refresh query	– 1 query per write, everywhere; also eliminated a whole class of async lazy-load crashes
Analytics snapshot seeding moved to a post-response background task	– 2 to – 4 queries off the user-facing critical path
Connection liveness ping disabled (redundant behind pgbouncer)	– 1 query on <i>every single request</i>
Five new hot-path indexes (action tables, list filters, submitter e-mail)	Seq-scans → index scans on the busiest lookups

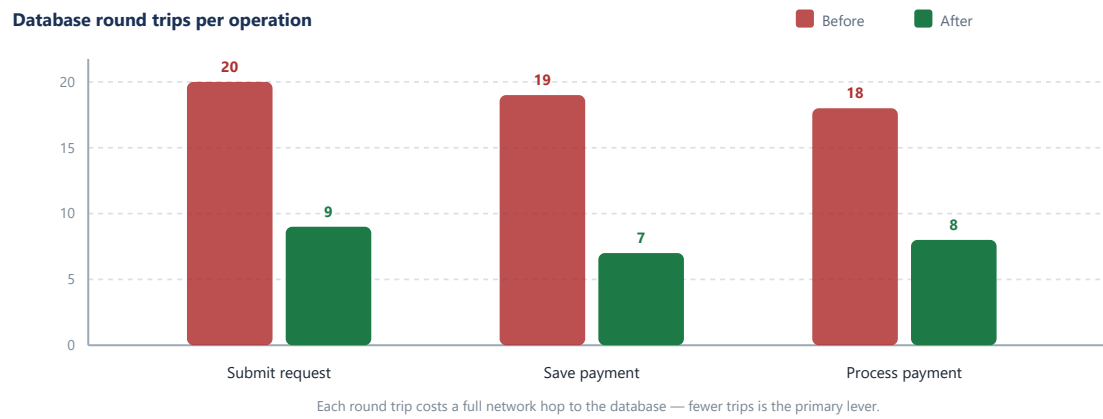


Figure 4 — Database round trips per hot-path operation, before vs after the optimisation release.

10.3 Root cause #3 — frontend overhead

- **Auth check per navigation** was a network call to the auth provider; replaced with a local cookie/JWT check — 100–400 ms saved on every page change.
- **Refetch storms:** every button click invalidated the entire request cache (6+ queries refired). Invalidation is now scoped to exactly the affected list and record.
- **Optimistic updates:** approve, reject, hold, resume, cancel, reopen and process-payment flip the UI instantly and reconcile with the server in the background — perceived latency for these actions is now effectively zero.
- **Bundle diet:** the AI chat (and its markdown renderer) is lazy-loaded; dev tooling excluded from production builds.

10.4 Root cause #4 — background waste

- Both server workers ran their own scheduler, refreshing all ~600 analytics snapshots twice every 30 minutes with one commit per row. Now: a single elected scheduler, finished/cancelled records skipped, and commits batched 100-at-a-time.
- Responses gzip-compressed (large list payloads shrink ~80%+); reference data served with 5-minute HTTP caching.

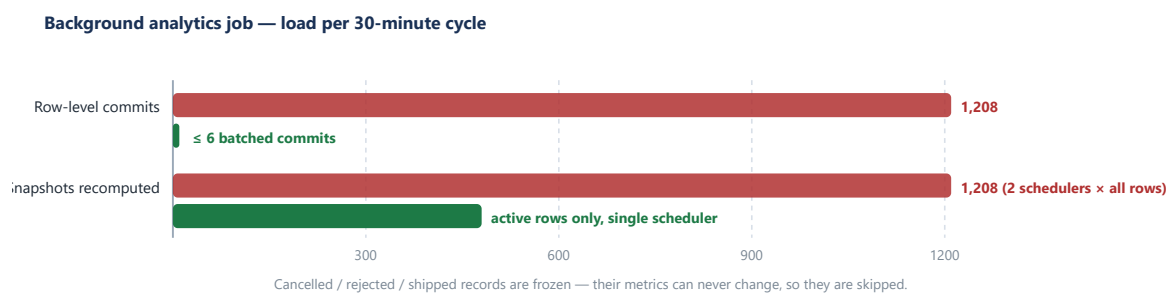


Figure 5 — Scheduled analytics refresh: duplicate schedulers eliminated, frozen records skipped, per-row commits batched.

11 Reliability & Error Handling

11.1 The "phantom CORS error" — and its permanent fix

Early in production, every backend crash surfaced in browsers as a misleading *"blocked by CORS policy"* network error, hiding the real cause. The root issue was architectural: the framework's default catch-all exception handler sits *outside* the CORS middleware, so error responses lost their CORS headers and browsers refused to read them.

Permanent fix: a custom error-envelope middleware placed *inside* the CORS layer guarantees that any unhandled exception returns structured JSON *with* CORS headers. Users now see the actual error message; no failure can masquerade as a network problem again.

11.2 Other hardening shipped

- Business-rule violations raised by database triggers are translated into clean, human-readable 400 responses (the trigger's own message) instead of blank 500s.
- Manually-constructed validation errors anywhere in the codebase are globally mapped to proper 422 responses.
- Malformed date filters on analytics and reports no longer crash — they are dropped gracefully.
- A full role-flow audit verified every button every role can see maps to an action the backend permits — eliminating dead buttons, stranded records (e.g. reopened requests with no path back to the queue), and broken landing redirects.
- Structured JSON logging throughout; deploy history and operational runbook maintained per release.

12 Analytics & Business Intelligence

12.1 The analytics engine

Every request carries a computed snapshot: **grace ETD** (estimated departure + configurable grace days), **ETD-grace overdue days**, **payment-to-ship days**, **payment-to-request days**, **cost-of-fund exposure** (configurable annual rate applied beyond a grace window), and a derived **risk status** (on-time / delayed / defaulted). Snapshots refresh on every relevant write (post-response) and via the 30-minute scheduler; grace days and rates are Super-Admin configuration, not code.

12.2 Dashboards

- KPI tiles (overdue counts, exposure totals) with permission-gated sections per role.
- Delay buckets, breakdowns by merchandiser / vertical / customer, cross-filtered drill-down tables, per-supplier analytics pages.
- Month-on-month and year-to-date trend charts with a year selector.
- Non-Performing-Assets panel: overdue suppliers (auto/manual flag provenance, max overdue days) and merchandiser performance.

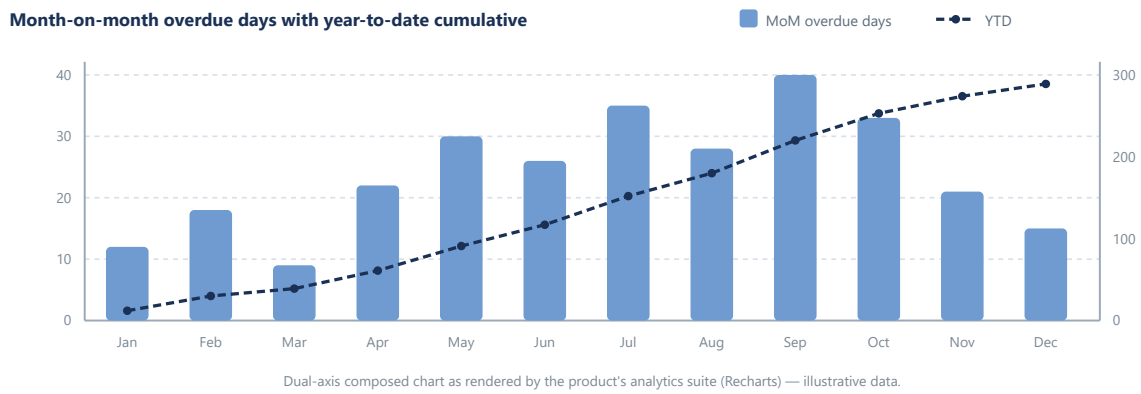


Figure 6 — The month-on-month / year-to-date trend visualisation shipped in the Analytics Overview (available for both overdue days and cost-of-fund, with a year selector).

12.3 AI assistant

A chat panel available on every dashboard translates questions like "which suppliers have the highest overdue exposure this quarter?" into read-only SQL against the live schema, executes it with an enforced row cap, and returns a natural-language summary with the underlying data. Access is role-gated and every AI query is audit-logged.

13 Excel vs. Platform — Outcome Comparison

Dimension	Excel workbook (before)	Advance Deposit Tracker (now)
Access control	None — full file or nothing	5 roles, per-endpoint + per-row + per-field enforcement
Approval workflow	Free-text status cells, unenforced	13-transition state machine enforced at 3 layers; HoM gate for risky suppliers
Audit trail	None	Field-level audit log for every change, with actor and timestamps
Concurrent use	Lost updates, e-mailed copies	Safe concurrent writes; advisory-locked numbering; record locking after payment
Supplier risk	Tribal knowledge	Auto-flagging on overdue shipments; 128 at-risk suppliers surfaced on day one; approval gate on flagged suppliers
Analytics	Manual formulas, frequently broken	Self-computing snapshots, dashboards, trends, NPA, cost-of-fund — refreshed automatically
Data quality	533 free-text payment terms; inconsistent dates/currency	Curated catalogues, typed and validated fields, deduplicated masters with unique indexes
Intake	Rows typed into the sheet	Validated in-app form + public form with supplier auto-fill and risk interception
Reporting	Copy-paste into new sheets	One-click role-scoped Excel / CSV / PDF exports (18 columns)
Querying	Filters and pivot tables	Plain-English questions via the AI assistant, plus drill-down dashboards
Recoverability	Whatever the last saved copy was	Managed Postgres backups; soft deletes; reopen workflow
Responsiveness	—	Sub-second interactions after the performance programme; instant optimistic feedback on all actions
Cost-of-fund exposure	Recomputed by hand when someone remembered; silently wrong as formulas broke	Computed automatically per request to the client's exact specification (Grace ETD → actual shipment date), frozen at shipment, identical on every screen, locked in by automated tests
Mistake prevention	Any cell could be typed over	40+ enforced guard rails: role gates, ownership checks, status rules, duplicate protection, terminal states, validation — refusals with human-readable reasons
Institutional memory	Lived in people's heads	Formula & Calculation Reference + this case study document every rule on paper for

		sign-off and hand-over
--	--	------------------------

14 Appendix A — Delivery Timeline

Phase	Releases	Scope
Foundation	Deploys 1–2	Core platform live: request CRUD, payment queue, analytics seeding fixes, payment-status dropdown, full report exports.
Change-request batch	Deploys 3–5	Client CRs: fixed deposits per supplier, required ETD, payment terms, MoM/YTD charts, master-data dedup + unique indexes, KPI tally fix.
Catalogue & UX	Deploys 6–9	Payment-terms master + admin CRUD, searchable combobox, filter persistence, 601-row Excel data migration, analytics recalculation.
Risk & approval	Deploys 10–13	Head-of-Merchandiser role, flagged-supplier override flow on both forms, HoM queue + detail pages, NPA panel, auto-flagging.
Hardening	Deploys 14–18	DB trigger alignment, double-submit protection, error-envelope middleware (phantom-CORS fix), full role-flow audit fixes, invoice-format rule relaxed per client, payment-terms consolidation to 33.
Performance	Deploy 19	Region relocation to Singapore, round-trip reduction programme, single-scheduler election, gzip + caching, optimistic UI, scoped cache invalidation, hot-path indexes.
UX refinement	Deploys 20–22	Supplier-risk overview shows supplier names (API serialization fix) with click-through per-supplier drill-downs; overview slimmed (outstanding column removed); design-system unification — admin toggle pills and audit-log action tags moved from ad-hoc pastels onto the navy token palette.
Financial precision & resilience	Deploys 23–25	Dynamic Cost of Fund per client specification (Grace ETD → actual shipment date, day-by-day accrual, frozen at shipment) with the client's worked example as an automated test; force-recalculation covering shipped records; Formula & Calculation Reference published; shared form-component hardening and self-recovering error boundaries.

Full per-file change logs for every release are maintained in the internal deployment history.

15 Appendix B — Glossary

Term	Meaning
Advance deposit	Partial payment made to a supplier before goods ship, typically a percentage of the invoice.
ETD / Grace ETD	Estimated time of departure; grace ETD adds a configurable buffer before a shipment counts as overdue.
Cost of fund	Financing cost of capital locked in an unshipped advance, computed at a configurable annual rate beyond a grace window.
Flagged / defaulted supplier	A supplier with an active default record — manual (finance) or automatic (overdue shipment) — requiring HoM approval for new advances.
NPA	Non-Performing Assets — advances whose shipments are overdue; grouped by supplier and merchandiser.
Snapshot	The per-request computed analytics row (overdue days, cost of fund, risk status).
Record lock	Immutability applied to a request once its payment is processed.
RLS	Row-Level Security — PostgreSQL policies restricting which rows a database role may read or write.
Optimistic update	UI pattern: reflect an action instantly, reconcile with the server response, roll back on failure.

Advance Deposit Tracker — Engineering Case Study · v1.2 · July 2026 · Prepared for company archives. System of record: Railway project (backend + frontend, Southeast Asia) · Supabase PostgreSQL (ap-south-1).